AgentWorkShop: A Modular Task-to-Device Integration Framework for Industrial Agents

***Abstract*—Industrial agents must translate task goals into operations over scene-specific equipment and process signals. We present AgentWorkShop, a modular framework connecting task entry, operation dispatch, configured device access, and process evidence. Registered harnesses bind agent engines to task context, while direct representational state transfer (REST) calls enter the same named-operation dispatcher. Scene blueprints map semantic signals and controls to drivers and define targets and process guards; owning services retain command receipts and subsequent observations. We evaluated the shared execution flow in a route study across four entry paths and two simulator scenes, with each route operating through its configured data-acquisition (DAQ) and write-control (DCW) nodes; a mock-state approval-context test rejected stale approval tuples before software dispatch, and a blueprint replay instantiated one documented coating-line configuration in simulation. We also analyzed four archived simulator missions—injection molding, biaxially oriented polyethylene terephthalate film, continuous annealing, and anaerobic/anoxic/oxic wastewater treatment—each a scripted operation–response–judgment cycle whose reported terminal readings were inside its target band (requests are not device readbacks). The route results establish record linkage across the tested entry paths; the archived missions illustrate scenario-specific feedback and are not pooled with the route evidence.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, semantic data integration.**

I. Introduction

INDUSTRIAL-AGENT deployments translate task-level goals into operations over production signals and equipment. Each operation depends on the active line, product, recipe, run, and device mapping. A reliable software path must therefore connect task context to device access and preserve the sequence from command issue to process response.

Recent work explores end-to-end automation, control-sequence generation, modular factory skills, and robotic-line control [1]–[4]. Other studies address programmable logic controller (PLC) code generation, context-aware control, resource discovery, shop-floor coordination, digital-twin fault handling, and natural-language machine access [5]–[10]. Recent platforms further connect proposals to Asset Administration Shell (AAS) models and signed approval, validate actions against process and instrumentation diagrams (P&IDs), or organize sensor–manufacturing execution system (MES)–enterprise resource planning (ERP) evidence streams [11]–[13]. Together, these efforts provide valuable methods for agent reasoning, production context, and operation constraints.

An integration question remains central to deployment: how should a common task runtime accommodate different agent engines and direct operation calls, while translating scene-specific signals into device actions and retaining the resulting process evidence? AgentWorkShop addresses this question through a shared operation path. Registered harnesses bind agent engines to task context; direct REST calls reach the same named-operation dispatcher. Scene blueprints define semantic signal mappings, targets, and guards, while configured drivers connect operations to device nodes.

Fig. 1 summarizes this control path. Task goals and production context inform an operation proposal. Shared checks, route guards, configured approval, and a pre-dispatch context recheck precede writes through the device write-control (DCW) subsystem. Command receipts and subsequent data-acquisition (DAQ) observations remain distinct records, so the software can relate an issued action to the process response.

AgentWorkShop implements this path as a modular task-to-device framework. Task and chat requests enter the collaboration runtime, where harness adapters connect different agent engines to production context, analysis services, DAQ/DCW nodes, and protocol-specific drivers. Command-line, web, software development kit (SDK), Representational State Transfer (REST), Model Context Protocol (MCP), and Agent-to-Agent (A2A) interfaces serve different callers. Scene blueprints and channel profiles define the available signals, objectives, guards, tools, and agent bindings. Fig. 2 shows these architectural relationships, and Fig. 3 follows one registered task through the execution path.

This paper makes two contributions. First, it presents a modular architecture that coordinates task entry, harness adapters, named operations, shared checks, configured device access,

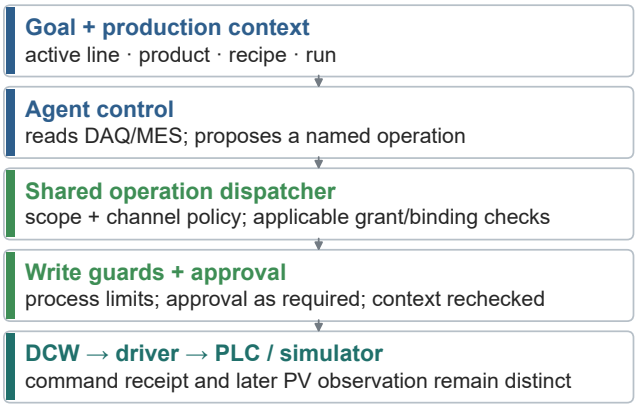


Fig. 1. Agent-control path for a configured industrial task. The agent uses DAQ and MES context, passes shared operation checks, and routes writes through process guards, conditional approval, and a pre-dispatch context recheck. DCW and configured drivers provide device access; command receipts and later process observations remain separate. Scene configuration supplies targets and guards, while the application provides its optimization policy.

DAQ/DCW node bindings, and service-owned evidence. Second, it evaluates the shared execution path through a route study across four entry paths and two simulator scenes, a mock-state approval-context test, and a blueprint onboarding replay. A separate archived seed-42 record, outside that evaluation, illustrates the operation–response–judgment sequence across four process objectives. The route study checks entry-specific records and node bindings; the approval test isolates the tuple comparison; and the onboarding replay instantiates one documented coating line in simulation.

Section II reviews related work. Section III presents the design rationale, architecture, and execution flow. Section IV explains the core module functions and their integration. Section V reports the route study, approval test, onboarding replay, and archived optimization record. Sections VI and VII interpret the findings and conclude the paper.

II. Related Work

This section relates AgentWorkShop to plant-facing industrial agents and to production abstractions for context, tool access, and operational constraints. Table I is a focused functional comparison, not an exhaustive survey. It includes studies discussed below that report plant-facing actions, manufacturing task execution, or production-context and operation-evidence mechanisms. The table compares two aspects of the integration contract: whether task and direct-operation entries share a named handler, and how operational evidence is associated. Entries describe reported features; "not reported" does not imply absence, and evaluation descriptions retain each source's metrics.

A. Industrial Agents and Plant Integration

Industrial-agent systems span end-to-end automation control, supervisory services, programmable logic controller (PLC) interfaces, manufacturing coordination, and operator-facing machine access. Xia et al. present an LLM framework for end-to-end automation control [1]; Huang et al. focus on automated control generation for industrial cyber-physical systems [2]; Biskupovic et al. study agent-based supervision [14]; and Agents4PLC and InstructMPC address PLC code generation and human–LLM control [5], [6]. Manufacturing studies consider resource-capability exploration [7], shop-floor coordination [8], and digital-twin fault handling [9]. Natural-language machine access [10] and real-time LLM control of a robot-operated line [4] extend this work to operator interfaces and robotic-line control.

Xia et al. connect cognitive, digital-twin, and physical-automation layers and organize twin information as state snapshots, event messages, and plan sequences [15]. Other nearby designs expose factory-module operations through MCP with state-based tool filtering, constrain manufacturing plans, or compose semantic recipes from OPC Unified Architecture (OPC UA) skills [3], [16], [17]. Several recent systems also combine agent execution with approval, industrial data, or audit records. Graul et al. ground proposals in the Asset Administration Shell (AAS) and route approved actions through OPC UA Methods, with signed approvals and audit logging [11]. Schall separates advisory language-model agents from process and instrumentation diagram (P&ID)-grounded verification and execution, and evaluates structural checks across five simulated process scenarios [12]; its context includes supervisory control and data acquisition (SCADA) and standard operating procedure (SOP) information. Lin and Chen build a governed sensor–MES–ERP digital thread for closed-loop monitoring and control on real manufacturing records [13]. These works address asset-specific approval, process validation, or closed-loop analytics; they motivate an integration layer that connects registered tasks and direct-operation calls to shared named handlers while retaining route-specific records. Table I positions the implementation mechanisms alongside reported evidence.

The operation checks complement safety-aware decision support [18] and operate alongside the plant's configured process protection.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also depends on abstractions that are not specific to agent architectures. The Model Context Protocol (MCP) specifies interoperable tool discovery and call mechanisms [19]; the operation-level scope, policy, grant, and binding checks discussed here are AgentWorkShop's own dispatcher rules, distinct from MCP transport authorization. In parallel, ISA-88 and ISA-95 describe recipes and production operations [20], [21]. Run-to-run methods connect process observations to successive production runs [22]. Human–automation research also formalizes the allocation of information acquisition, analysis, decisions, and actions between people and automation [23]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [24], [25]. Together, these standards and agent-level mechanisms provide abstractions for production context, tool interaction, and information flow. AgentWorkShop brings these abstractions into one execution path: task and direct-operation calls converge on named handlers, scene mappings connect semantic controls to configured drivers, and owning services retain command receipts and process observations. Section V examines this path through a route study, an approval-context test, a blueprint replay, and four archived optimization missions.

III. System Design

AgentWorkShop is designed for industrial tasks whose agents, production contexts, interfaces, and process signals vary between deployments. Its architecture organizes these variations around shared task and operation services, checks each proposed operation against the active production context before device routing, and retains route-specific evidence that distinguishes a command receipt from a measured process response.

A. Design Rationale and Requirements

Five requirements—scene variability, harness variability, operation authority, outcome observability, and extensibility—shape the design. Scene variability is represented by blueprints

TABLE I
REPORTED SHARED-ENTRY HANDLING AND EVIDENCE STRUCTURE IN SELECTED PLANT-FACING STUDIES. SHARED HANDLING MEANS THAT TASK AND DIRECT-OPERATION ENTRIES RESOLVE TO THE SAME NAMED HANDLER; "NOT REPORTED" IS NOT A CLAIM THAT A SYSTEM LACKS THE FEATURE. EVALUATIONS RETAIN SOURCE-SPECIFIC SCOPE AND CRITERIA.

System	Shared handler for task and direct entries	Reported operation and evidence structure	Reported evaluation
Xia et al. [15]	Not reported	Cognitive, twin, and physical layers; twin snapshots, event messages, and plan sequences	Robotic automation and process-simulation cases
Köhle et al. [3]	Not reported	Injected live state and state-based filtering; MCP-to-OPC UA operations and Message Queuing Telemetry Transport (MQTT) coordination	Nine simulated production challenges
Vieira da Silva et al. [16]	Not reported	Constrained multi-step plans over MCP manufacturing functions	Laboratory multi-step process
Köhle et al. [17]	Not reported	Semantic recipes composed from OPC UA skills; virtual commissioning	Unity assembly-workstation proof of concept
Graul et al. [11]	Not reported	AAS-grounded proposals, signed human approval, PLC-validated OPC UA Methods, and immutable audit log	Two laboratory PoCs; single scripted runs
Schall [12]	Not reported	P&ID-, SCADA-, and SOP-grounded proposal verification with gated execution	Five simulated scenarios; 43 proposals and 50 LLM runs
Lin and Chen [13]	Not reported	Governed sensor–MES–ERP digital thread with SOP-, risk-, and capability-aware action gating	25,275 real records; operational metrics
This work	Shared dispatcher; scene-bound DAQ/DCW nodes	Route-specific records linked to process observations; approval-context check	Four entry paths in two simulator scenes; one blueprint replay; four archived optimization missions

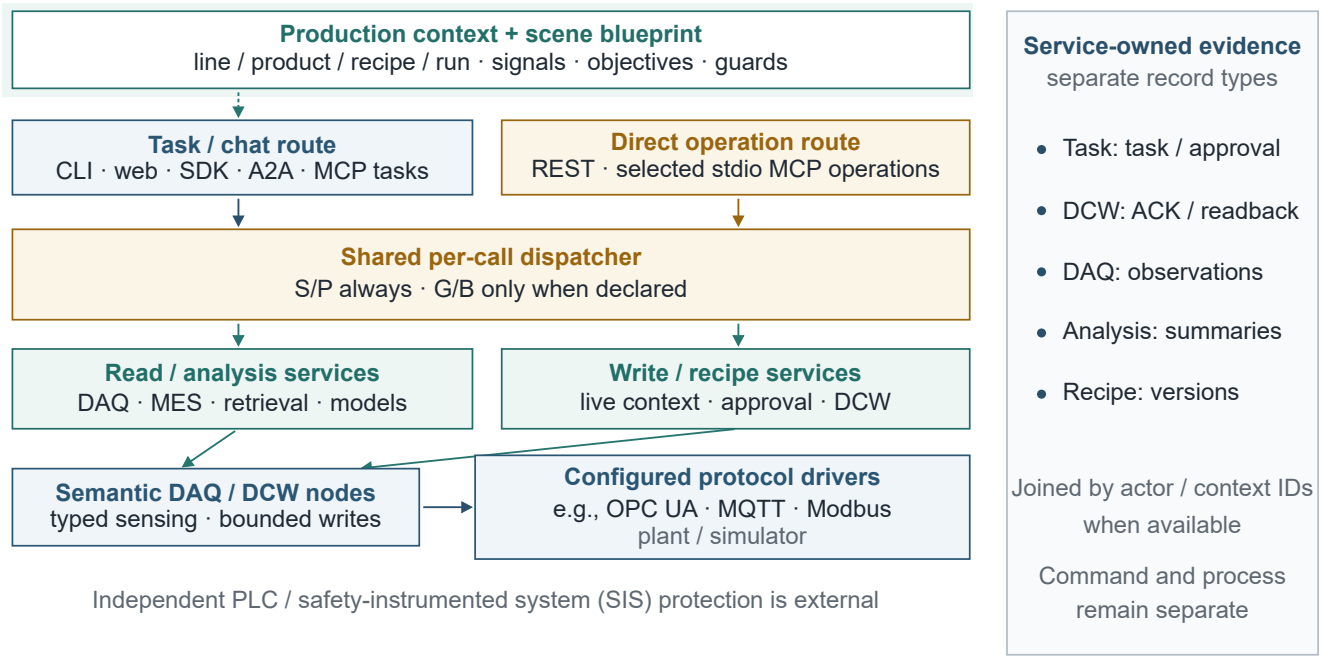


Fig. 2. Architecture and record ownership. Task/chat routes enter the collaboration runtime; direct REST and stdio-MCP calls enter shared dispatch checks without creating a task. One dashed link marks configuration. Write routes add approval and interval admission; recipe dispatch omits the previous-recipe window. The shared dispatcher always checks scope and channel policy, with grant and resource-binding checks applied when required by the operation. Owning services retain distinct records; plant controllers and safety systems retain equipment interlocks.

and semantic driver mappings; harness variability by registered adapters; operation authority by shared dispatch checks and route-specific guards and admission; outcome observability by service-owned task, receipt, and observation records; and extensibility by registered interfaces for agent engines and device drivers. Tool visibility is separate from write authorization, and

actuator writes converge on the shared DCW admission point.

A scene blueprint bundles the process preset, signal inventory, DAQ/DCW driver mappings, and target and guard rules; judged operation records and later observations remain distinct evidence.

During setup, the integrator binds a validated blueprint to the active line–product–recipe–run tuple, maps semantic nodes to drivers, and declares target bands and guards. At runtime, the dispatcher checks current authority before the handler issues a command; the handler returns a receipt, and DAQ records subsequent observations.

B. Layered Architecture

Fig. 2 summarizes the access routes, shared dispatch path, production services, semantic I/O, configured drivers, and service ownership of records. Task and chat routes enter the collaboration band, which contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. A *task route* is an execution path through this runtime; a *harness adapter* connects an agent engine to its task execution interface. Direct REST tool calls and the stdio MCP operation bridge bypass task execution in that band (the upper-right direct-operation entry in Fig. 2) and enter the shared governance block without creating a task or starting a harness turn; MCP management and task methods remain on their service routes. Direct calls resolve current route state at invocation, without an agent-turn prompt or a task record. The governance block projects the operation catalog and applies the per-call checks; write routes add the approval card and the admission interval. The context block supplies the active line–product–recipe–run tuple to the gate and scene maps, objectives, and guards to the handlers. The dashed link from that block in Fig. 2 marks this configuration relationship; the runtime assembles prompt-time context from the same records. The service band, behind a route-handler interface that selects an adapter, provides acquisition, write control, analysis, retrieval, and optional model services. Field I/O resolves semantic nodes to built-in or plugin drivers for a plant or simulator.

Each band has a distinct responsibility: task/chat routes enter the runtime, direct operations enter the dispatcher, and both use shared checks. Harness adapters supply agent-specific interaction, semantic nodes hide protocol details, and the driver registry provides the device-facing extension point. Owning services retain task, command, observation, and version records separately and link them to actor and production context.

C. Cross-Module Execution Flow

Fig. 3 summarizes a task turn from entry through operation execution, with route-specific evidence retained by owning services. Registered harness adapters receive a scoped tool catalog during setup and supported registry refreshes; the runtime assembles prompt-time production context from current records for each agent turn. For each returned tool call, the shared dispatcher rechecks invocation scope and channel policy and, for industrial operations, current grant and binding conditions before handler selection. The selected handler then applies route-specific guards, active-run checks, approval revalidation, and device-I/O steps as applicable. Command receipts and later process observations remain distinct records.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a, c, u) = \bigwedge_{X \in \mathcal{C}(u)} X(a, c, u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, runtime grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S, P, G, B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is omitted only for an explicitly grant-exempt industrial tool, and B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. Once these predicates pass, the selected route applies its guards and approval, then rechecks authority, context, and applicable limits before device dispatch.

For direct REST invocation, the service establishes a and c before applying these checks: an agent token must match the requested agent identifier, or an authenticated user must be the channel owner or an administrator. The manager loads the channel-agent record and rechecks scope and policy; industrial operations then pass the shared grant and binding checks. Handlers resolve live production state from request identifiers and owning services.

Fig. 4 gives reference pseudocode summarizing the registered-harness task path. Direct REST tool invocation bypasses the task runtime and harness but enters the same shared dispatcher. Route-specific timing, approval, and evidence rules are detailed in Section IV.

IV. Module Functions and Integration

The design in Section III is realized through three cooperating functions: task and context assembly, shared operation dispatch, and controlled device access with route-specific evidence. We follow one agent action through them.

A. Task, Harness, and Context

Task and chat entry points create or join a workspace channel. The task runtime manages membership, task state, and shared execution context; a harness registry adapts each supported agent engine to a common execution interface. At each agent turn, the prompt builder assembles the task, active scene description, production context, channel profile, agent binding, and retrieved memory. This keeps engine-specific interaction behind the harness interface while preserving the current line and operation context for the agent.

Direct REST and selected stdio-MCP operation calls reach the same dispatcher without a task or harness turn. REST authentication and identity resolution follow Section III-C; each handler reads live route state at invocation. Handlers

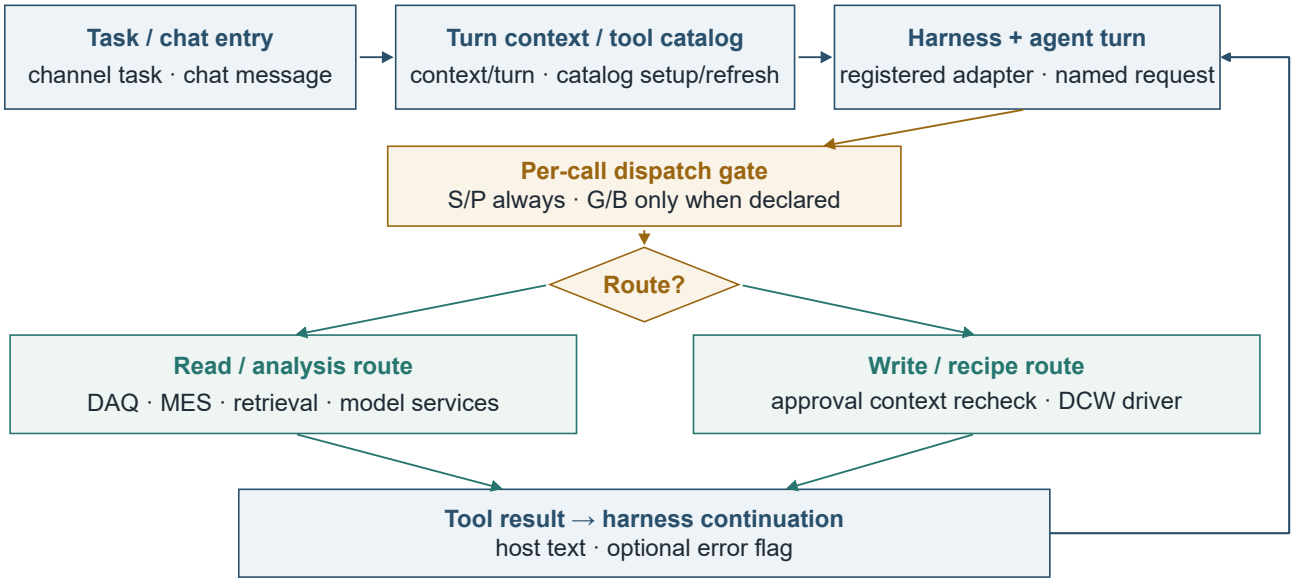


Fig. 3. Registered-harness task flow. The runtime invokes the adapter with a scoped catalog. Scope and channel policy are checked for every call; grant applies to non-exempt industrial operations, and binding applies when resource references are declared (figure label: S/P always, G/B only when declared). Failed checks return tool errors. The read path retrieves data. The write path applies route guards, approval and context revalidation, DCW admission, and receipt grading; a later process observation is separate. Results return to the harness. Device-level interlocks remain with the configured plant-control layer.

Require: task q, channel c, bound agent a, registered scoped catalog T
Ensure: task result; any retained evidence remains with its owning service
$x_t \leftarrow \textsc{ResolvePromptContext}(c, q)$
$m \leftarrow \textsc{RetrieveMemory}(q)$
$p \leftarrow \textsc{AssemblePrompt}(q, x_t, m)$
$h \leftarrow \textsc{InvokeRegisteredHarness}(a, p, T)$
for all tool request u returned by h **do**
  $g \leftarrow \textsc{CheckScopeAndChannelPolicy}(a, c, u)$
  if not g **then**
    $\textsc{EmitToolErrorToHarness}$(h,u); **continue**
  end if
  if u is an industrial operation **then**
    $v \leftarrow \textsc{CheckApplicableGrantBinding}(a, u)$
    if not v **then**
      $\textsc{EmitToolErrorToHarness}$(h,u)
      continue
    end if
  end if
  $k \leftarrow \textsc{SelectHandler}(u)$
  $r \leftarrow \textsc{ExecuteRouteHandler}(k, a, c, u)$ ▷ handler resolves live state and may persist route-specific records
  $\textsc{EmitDeclaredToolResultToHarness}$(h,r)
end for
return task state and tool results

Fig. 4. Reference pseudocode for the registered-harness task path. Direct operation calls use the shared dispatcher from their own entry point. Each route persists its records through the owning service and returns its declared response to the harness.

receive the resolved actor, channel, and operation arguments through the same interface for either entry route; records are keyed by the identifiers supplied in each request. For either entry path, prompt context and tool-catalog visibility describe available information and capabilities, while dispatch-time checks determine write authority.

B. Shared Dispatch and Device Admission

The dispatcher is the shared operation boundary. It checks invocation scope and current channel policy, then verifies the agent binding, runtime grant, and ownership of referenced resources before selecting a handler. In the notation of (1), scope and policy apply to every call; grant is skipped only for a grant-exempt tool, and binding applies only when references are declared. Handlers revalidate route-specific state at execution time, using the current policy and grant for each invocation. Scene blueprints define device and signal maps, objectives, and process guards. Capability profiles and tool policies narrow available operations, while bindings constrain accessible nodes, controls, and recipes. Optional DAQ, MES, analysis, retrieval, optimization, and digital-twin services are invoked through registered handlers. The application supplies its optimization strategy while the framework coordinates operation execution.

Actuator writes converge on the shared DCW admission point. For an online node write, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{2}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are the baseline, active-product, and active-recipe limits; the subscripted limits here are distinct from the policy and binding predicates of (1). Missing optional limits impose no restriction;

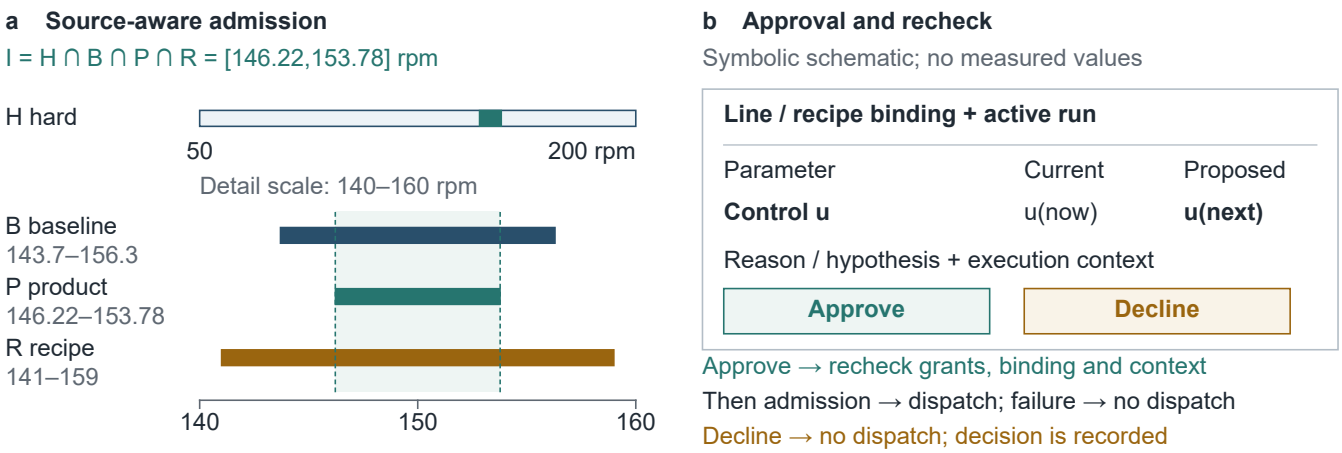


Fig. 5. Admission and approval boundaries. (a) An illustrative rotational-speed configuration in revolutions per minute (rpm): online node interventions use the four-layer interval of Eq. (2), whereas recipe dispatch omits the previous recipe window. (b) Symbolic human approval binds a proposed action and its context tuple; execution rechecks the active line–product–recipe–run context, authority, and applicable limits before device dispatch.

an empty intersection rejects the request and identifies the excluding layer. Recipe dispatch validates the replacement recipe against the hard, baseline, and product limits rather than the previous recipe window. Step, interval, and hold-window checks also constrain each route. These software checks bound commands alongside independent programmable-logic-controller and safety-instrumented protection.

C. Approval, Execution, and Evidence

Routes add timing and approval checks appropriate to their operation. Online node writes enforce a 60 s minimum spacing; recipe trials use a per-line cadence (300 s by default), and recipe intervals are anchored to approval. Manual approval binds an actor and proposed operation to candidate values, applicable limits, expiry, and the active line–product–recipe–run tuple. After approval, the handler rechecks authority and compares the current tuple with the approved tuple before entering DCW dispatch. Single-parameter waits expire after 180 s by default; recipe-dispatch approvals use a 30-minute window. A changed recipe definition requires a new `recipe_apply` proposal.

Fig. 5(a) illustrates the four-layer value interval in Eq. (2) using an illustrative rotational-speed configuration in revolutions per minute (rpm). Panel (b) shows a symbolic approval followed by authority and context rechecks; its values illustrate the approval sequence.

A `recipe_trial` illustrates the operation sequence: the handler checks the bound recipe, active run, candidate parameters, and route timing; if required, it requests approval; after revalidation, it submits a bounded batch through the configured driver and returns a per-parameter receipt. The caller obtains later DAQ observations separately and may then update, apply, or roll back a recipe. A trial neither saves nor applies a recipe version. The driver registry maps semantic controls to configured protocol drivers, keeping protocol details out of agent operation arguments.

Receipts and observations answer different questions. A command result is graded as readback-verified, transport-acknowledged, or unverified; a readback mismatch can trigger up to three reads at 2 s intervals. Recipe batches retain per-parameter verification outcomes. DAQ stores subsequent timestamped process observations, while recipe versions, approvals, and task records remain service-owned records linked where a route supplies the corresponding identifiers. Command receipts document device execution, while subsequent DAQ observations establish the resulting process response. This distinction defines the evidence chain evaluated in Section V.

V. Experimental Evaluation

The experiments examine four questions: (Q1) whether the tested execution routes retain the records required for each entry path—task, approval, receipt, and sample links where a task exists—across two configured simulator scenes and whether each route operates through its bound DAQ and DCW nodes; (Q2) whether one documented blueprint-onboarding workflow completes on a fresh application home; (Q3) whether a stale approval context is rejected before software dispatch; and (Q4) what process samples accompany the tested routes. These outcomes are kept separate. All plant-facing checks used software simulators; none involved a physical plant or independent process replication. Sections V-B to V-D report the current-version outcomes; Section V-E reports the separate historical optimization record. The evidence is complementary and is not pooled into a single score.

A. Test Design and Protocol

For a registered task, the integration criterion is a task linked to its approval, readback-verified receipt, and a run-tagged sample. A direct call has no task edge; its criterion is a verified receipt/readback and a run-tagged sample. The fixed-call replay checks record links, not task completion; the context guard is assessed by whether a changed line, product, recipe, or run is refused before a driver write. Table II summarizes the four studies and their inference limits.

The route-continuity study exercised four entry paths on the v0.8.0 application: registered-harness tasks, a fixed-call harness replay, direct REST, and a fixture-backed command-line (CLI) task. Two simulator scenes covered cast film

(screw speed over OPC UA; thickness over MQTT; target 48–52 μm) and continuous annealing (zone-3 setpoint and strip temperature over Modbus TCP; target 700–760 °C). Each scene blueprint declared its DAQ and DCW nodes, and the route configuration bound the agent and channel tool catalog to those nodes; the recorded reads and writes used the bound thickness/screw-speed and strip-temperature/zone-3 nodes. The registered-harness path ran three sequential tasks per scene on one persistent simulator; the registered adapter used a fixed cloud model (`glm-5.3-flash`) with a three-value prevalidated set per control. Each task used its own production and task context, while process state carried forward. After each verified receipt, the runner waited 20 s, probed and ingested one sample, queried it by active-run ID, and then checked task state, so the completion-to-sample latency was not recorded. The fixed-call replay, direct REST call, and CLI task ran once per scene in separate isolated environments.

The production MQTT queue and Timescale service were not used in the route-study runs; R22 ran isolated with an in-process queue and SQLite storage. Each sample was a single explicit driver probe stored and queried through the local SQLite fallback with online DAQ collection disabled; the route results therefore assess invocation and application-record handling, not the online DAQ pipeline. Fixture-backed routes used deterministic input; direct REST created no task and did not invoke an external model. All route-study approvals were synthetic administrator decisions.

A separate v0.8.3 onboarding replay used a fresh isolated application home and the documented C01 blueprint. The blueprint configured one Modbus/TCP device with four DCW setpoints and seven DAQ nodes, plus MES REST scalar, 48-point vector, table, and image forms. Channel construction bound the agent and tool catalog to those nodes. Dedicated local MQTT and Timescale services were online for DAQ checks; object storage was unavailable, so image artifacts used the local-disk fallback. This was one configuration replay, not a route-study trial.

The approval-context suite changed one field of the approved line–product–recipe–run tuple at a time for recipe apply, trial, and rollback with dispatch enabled: four fields for each of the three operations. A separate fault-injection case changed context after asynchronous grant revalidation but before the caller's final synchronous check. A test-only ablation removed only the tuple comparison while retaining the other checks. These outcomes were recorded in mock-driver state and are not process-level reliability estimates.

B. Route Results and Node Bindings

All six model-mediated registered tasks completed, and their task, approval, receipt, and sample links were verified. Cast-film settings of 125/130/125 rpm produced single 20-s samples of 45.69/46.69/45.69 μm; annealing settings of 745 °C produced 679.60/677.70/678.10 °C. Pre-task values were 55.82/45.69/46.69 μm and 675.40/678.30/676.90 °C, so five tasks started below their band and one above. The post-trial samples therefore reflect carried-over state and a single 20-s probe; they do not describe a response trajectory, steady state, or the quality of the model-selected settings. The recorded reads and writes used the bound thickness/screw-speed and strip-temperature/zone-3 nodes. The study did not inject an unbound-node request, so binding is observed as the precondition under which the routes succeeded, not as an ablated factor.

TABLE II
EXPERIMENT MATRIX. ALL ENTRIES ARE SOFTWARE-LEVEL CHECKS, NOT INDEPENDENT PROCESS REPLICATIONS.

Study	Design, observed result, and inference limit
Route continuity and binding (Q1)	Four entry paths in two simulator scenes; each route bound to the scene's DAQ and DCW nodes. Six model-mediated registered tasks verified task, approval, receipt, and sample links; all route samples were below target. One 20-s sample per task or call (12 total); no unbound-node negative test.
C01 onboarding and binding (Q2)	One documented blueprint: four DCW setpoints, seven DAQ nodes, four MES forms. Preflight, provisioning, channel construction, and the V1–V6 verifier completed; one 37–39 m/min simulator-confirmed write. One blueprint; unpinned build.
Approval context (Q3)	Four tuple fields for each of three recipe operations with dispatch enabled; one queued between-check mutation. Every injected one-field change was blocked before a mock write; an ablation that removed only the tuple comparison admitted six product/run changes. Mock state only.
Archived optimization (historical)	Four seed-42 scripted direct-node missions, one trajectory each. Terminal readings were in band; requests are not device readbacks. Not current-route or repeatability evidence.

The remaining three routes met their record criteria but differed in task linkage and model involvement. The fixed-call replay linked both tasks to approval, receipt, and active-run samples without asserting task completion; its samples were 46.37 μm and 674.80 °C. Direct REST returned verified receipts/readbacks and run-tagged samples; its samples were 46.37 μm and 675.20 °C. The fixture-backed CLI task completed both tasks and linked their samples; its samples were 46.38 μm and 674.40 °C. All three used 125 rpm and 735 °C, whereas the registered runs used 745 °C. The cast-film samples are similar across routes; the annealing samples differ by roughly 4–5 °C, consistent with the setpoint difference. These similarities are consistent with a shared handler, but separate environments and one sample per task or call do not establish route equivalence, process attainment, or reuse across scenes.

C. HITL Approval Context and Stale-Context Rejection (Mock State)

For a recipe write, the handler checks the bound recipe, active run, candidate parameters, and route timing; if human-in-the-loop (HITL) approval is required, it creates a card binding the actor, operation, candidate values, applicable limits, expiry (30 min for recipe dispatch; 180 s for single-parameter HITL), and the active tuple. After approval, the handler rechecks authority and grant, compares the current tuple with the approved tuple, and only then enters DCW dispatch. The queued context change was refused at the same comparison.

The production handler blocked every injected one-field mutation before a mock write or execution record, and the queued change was refused before `recipe_apply` dispatch. In the ablation, removing the tuple comparison allowed six

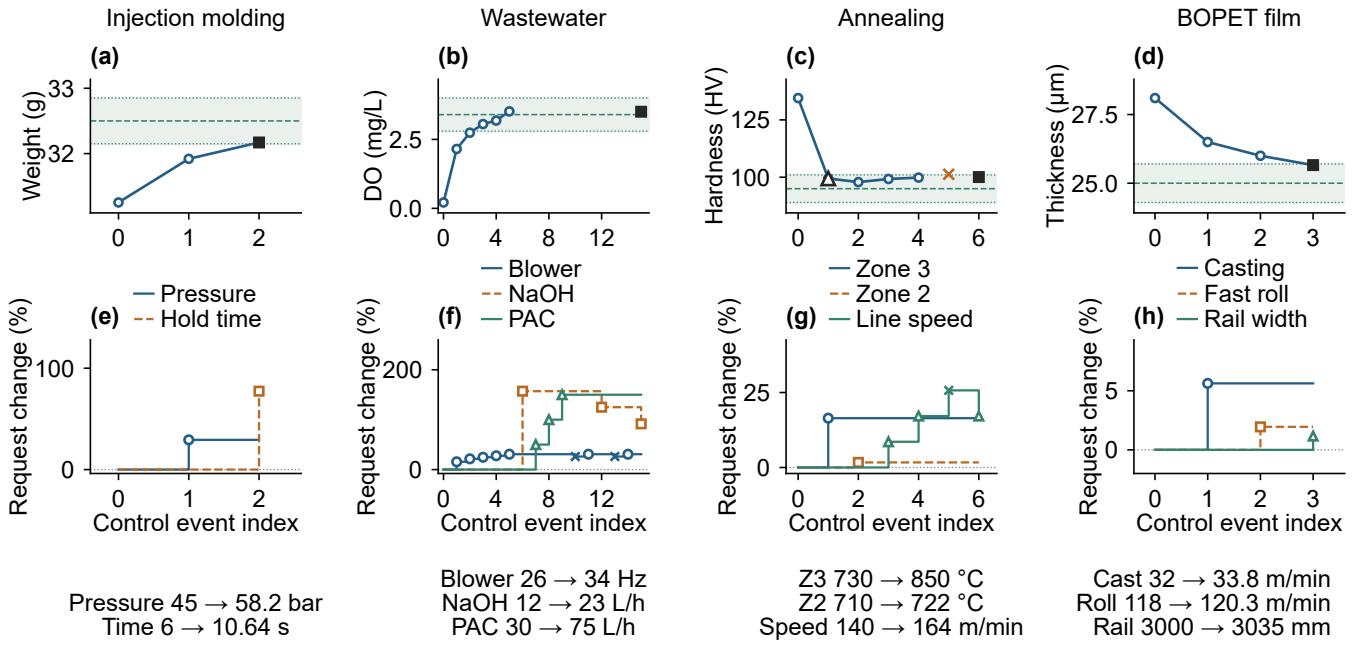


Fig. 6. Archived optimization records for four simulator scenarios (seed 42; one scripted trajectory per scenario). Upper panels show process-variable (PV) observations with scenario-specific target bands; lower panels show requested setpoint changes as percentages of each baseline, and event indices give order, not elapsed time. Bottom annotations give the first and last requested values. Black squares mark mission-final readings; for injection molding and BOPET they overlap the last observation. In wastewater treatment, later NaOH and PAC requests belong to pH and total-phosphorus stages, and the mission-final DO reading (3.50 mg/L) is not joined to the last trajectory observation (3.517 mg/L). Crosses mark reverted requests. In annealing, the open triangle is the 99.46-HV event without a linked optimization record; the cross is the 101.3-HV event-text value after a reverted 176 m/min candidate, not a structured trajectory observation. Requests are not device readbacks. These historical direct-node missions are separate from the route study and do not show current route performance or a common optimizer score.

product/run changes through; each produced two mock writes and an execution record, while the other checks still blocked line/recipe changes. The comparison is therefore the decisive check for product/run identity, whereas the other checks cover line and recipe changes.

The study is mock-state only. It does not demonstrate atomicity during network or device I/O, safe physical operation, operator effectiveness, or human-review performance. Synthetic approval also does not measure approval latency or error rates.

D. Blueprint Onboarding and DAQ/DCW Binding

The C01 blueprint instantiated the documented coating line from configuration objects: one Modbus/TCP device with four DCW setpoints and seven DAQ nodes, plus MES REST scalar, 48-point vector, table, and image forms. Channel construction bound the agent and tool catalog to those nodes. Its connectivity preflight, line provisioning, and channel construction completed with the expected objects and no failures, and all four MES forms returned records with the expected format and field dictionary. The V1–V6 verifier confirmed sample availability for all seven DAQ nodes, range rejection, legal write/readback, recipe mapping, channel/plugin checks, and zero writes to a protected demonstration line.

One smoke proposal used the test-only automatic-approval policy to change line speed from 37 to 39 m/min within the 28–48 m/min recipe window; the PLC twin confirmed the write. The approval was synthetic. The replay ran on a working-tree build whose source commit was not pinned, and it did not vary the profile or binding set between runs. These checks therefore establish one documented blueprint-to-simulator onboarding instance, not human-review performance, film quality, process attainment, arbitrary protocol coverage, configuration reuse, repeatability, or physical transfer. Supplementary File S1 includes the inputs, health and DAQ status, runner output, and complete V1–V6 log.

E. Archived Optimization Execution Records

The current route study collected one post-trial process sample per task or call. Fig. 6 instead shows four archived seed-42 optimization missions from an earlier scripted direct-node interface. Each mission iterates: observe the process variable, request a bounded setpoint change, apply it through the direct node, and observe the next process variable. The event index gives order, not elapsed time. The upper row plots observations against each scenario's target band; the lower row plots requested changes as percentages of each baseline; the annotations give the first and last requested values.

In injection molding, hold pressure and time increased from 45 to 58.2 bar and 6 to 10.64 s; measured weight rose from 31.25 to 32.17 g, ending 0.02 g inside the 32.5 ± 0.35 g window. Five wastewater blower increments from 26 to 34 Hz raised dissolved oxygen (DO) from 0.214 to 3.517 mg/L, inside the 3.4 ± 0.6 mg/L window; later NaOH and polyaluminium chloride (PAC) requests moved from 12 to 23 L/h and 30 to 75 L/h in separate pH and phosphorus stages. In annealing,

the zone-3 request rose from 730 to 850 °C without a linked optimization record; zone-2 moved from 710 to 722 °C and line speed from 140 to 164 m/min, producing 97.88–99.84 Vickers hardness (HV), and the terminal value was 100.08 HV in the 95 ± 6 HV window. For biaxially oriented polyethylene terephthalate (BOPET) film, casting speed, fast-roll speed, and rail width changed from 32 to 33.8 m/min, 118 to 120.3 m/min, and 3000 to 3035 mm; thickness fell from 28.10 to 25.66 μm, ending 0.04 μm inside the upper edge of its 25.0 ± 0.7 μm window.

These archived records illustrate scenario-specific feedback, not repeatability or current-route attainment. They also show that the archived scenarios can reach in-band terminal readings under scripted direct-node control, but they measure different process variables from the route study and do not bound the route's cast-film thickness or annealing strip temperature. The historical missions are not pooled with the route results.

F. Evidence Synthesis

The route study answers Q1 and Q4: across two preconfigured mappings, every tested task and direct-call contract retained its required records, every route used its bound DAQ and DCW nodes, and every route-linked sample was below target. The onboarding replay answers Q2: the documented C01 configuration completed on a fresh application home, produced the expected objects with isolated online MQTT/Timescale, and confirmed one simulator write. The mock-state comparison answers Q3: the tested context check blocked stale requests before software dispatch. Fig. 6 is historical optimization evidence and is not pooled with these results.

Together, these results distinguish command completion from process outcome: a verified readback confirms the setpoint path, whereas the later run-tagged sample indicates whether the process variable was inside its target band at the probe instant. The study establishes record linkage across the tested entry paths, that the routes used their configured bindings, stale-context rejection in mock state, and one simulated onboarding instance; it does not establish process improvement, general portability, or physical-plant performance. Supplementary File S1 includes the route inputs and logs, onboarding replay, historical records, and evidence/hash manifests. The model-mediated tasks are not deterministic replays, and the route samples are not independent replicates.

VI. Discussion

A. Interpretation

The route study supplies the current-version evidence. Registered tasks retain task, approval, receipt, and sample associations; direct REST retains operation, readback, and sample records without a task edge. Each route used the scene's bound DAQ and DCW nodes, so binding is observed as the precondition under which the recorded reads and writes succeeded. The two simulator scenes use different signal and protocol mappings, and the fixed-call and CLI routes separate task-path behavior from an external model decision. Similar post-trial samples are consistent with a shared handler; the annealing difference is consistent with the 735 versus 745 °C setpoints. But separate environments and one sample per task or call do not establish route equivalence.

The approval-context test isolates the tuple comparison under mock state: removing that comparison admitted product/run changes that the retained checks alone did not reject, while line and recipe changes remained blocked. The blueprint replay shows that one documented coating line can be instantiated from configuration objects with four DCW setpoints, seven DAQ nodes, and four MES forms, and that the verifier can check sample availability and write behavior without altering the protected demonstration line.

The four archived missions apply one operation–response–judgment workflow to distinct objectives: paired holding adjustments for molding, three-control thickness correction for BOPET, throughput under a hardness constraint for annealing, and staged aeration and dosing for wastewater. Most requested changes are followed by a recorded process observation; reverted candidates and the 101.3-HV event-text value are marked in the figure. The missions show how a common operation path accommodates different actuator structures and target types. They are historical scripted direct-node records, not current-route trials, and each is a single trajectory.

The process results bound the interpretation. Cast-film samples were 1.31–2.31 μm below the 48 μm lower bound, and annealing samples were 20.40–25.60 °C below 700 °C. Because five of six tasks started below their band, the post-trial shortfall is not evidence that the selected setpoints caused it; the one above-band start cannot be resolved with a single 20-s probe. The archived missions end inside their bands, but they measure different process variables and are not a controlled comparison with the route study.

Taken together, the evidence supports a software separation between task reasoning, operation resolution, scene mapping, and process observation; the archived missions show that this sequence survives changes in actuator structure and target type.

B. Scope and Limitations

Only the reported adapters and scenes were exercised. The route runs and the C01 replay used working-tree builds rather than pinned releases; the route study used the SQLite fallback because the production MQTT/Timescale backends were not used; and the approval evidence is mock state with synthetic approvals. The four archived missions are one seed-42 trajectory per scenario, generated through an earlier scripted direct-node interface, and are not current-route, repeatability, or physical-plant evidence. The study also does not establish adapter interchangeability, configuration reuse, normal online DAQ behavior, or a platform baseline. The feature comparison positions the implementation question; it does not show that AgentWorkShop is more efficient or more capable than another platform. A comparative study would need a shared task set, explicit integration-effort or record-completeness measures, and matched deployments. Operation restrictions do not establish security [26]–[28] or replace independent PLC/safety-instrumented system (SIS) safeguards [29]. Synthetic approval also does not measure human-review

performance [30], [31]. Scope and channel-policy refusals are not uniformly persisted; auditability is limited to the records and denial classes described in Section IV.

VII. Conclusion

AgentWorkShop connects industrial task workflows to scene-specific operations through shared dispatch, configured drivers, and service-owned process records. A route study across four entry paths preserved entry-specific records and used scene-bound DAQ and DCW nodes; a mock-state approval-context test rejected stale approval tuples before dispatch; a blueprint replay instantiated one documented coating-line configuration in simulation; and four archived simulator missions illustrated the operation–response–judgment sequence across distinct process objectives. The study establishes record linkage across the tested entry paths, stale-context rejection in mock state, and one simulated onboarding instance; it does not establish process improvement, configuration reuse, general portability, or comparative platform benefit.

References

[1] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[2] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[3] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[4] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. IEEE 64th Conf. on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[11] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[12] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[13] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[14] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[15] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[16] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[17] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *Proc. IEEE 8th Int. Conf. on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[18] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[19] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, version 2026-07-28, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[20] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[21] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[22] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[23] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[24] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[25] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[26] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[27] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[28] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[29] ——, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.

[30] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[31] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.